

Dokumentation – Cyber-Physische Systeme

Projekt Teil 1 – Sensorik & Datenübertragung mit ESP32

Name:

Nico Flöter

Klasse:

FI 24

Datum:

12. Dezember 2025

Inhaltsverzeichnis

- 1. Einleitung & Projekthintergrund
 - 2. Projektbeschreibung & Zielsetzung
 - 3. Arbeitsschritte
 - 4. Hardware & Komponenten
 - 5. Implementierung & Programmcode
 - 5.1 Einlesen von Temperatur & Luftfeuchtigkeit (DHT22)
 - 5.2 Bewegungsmelder (PIR) & LED-Signalisierung
 - 5.3 Lichtsensor (Fotowiderstand / LM393)
 - 5.4 JSON-Datenformat & Sekündliche Ausgabe
 - 5.5 OLED-Display-Ausgabe (SSD1306)
 - 5.6 Temperatur LED-Anzeige
 - 6. Gesamter Programmcode
 - 7. Testergebnisse & Fazit
-

1. Einleitung & Projekthintergrund

Im Rahmen des Lernfeldes *Cyber-Physische Systeme* soll ein Prototyp zur **Überwachung eines Verwaltungsgebäudes** entwickelt werden. Das System soll in unterschiedlichen Räumen die Raumtemperatur, die Luftfeuchtigkeit, die Lichtstärke und die Bewegung von Personen erfassen. Die gesammelten Daten sollen mittels Datenvisualisierung auf einem Dashboard in Echtzeit angezeigt werden.

Als Ausgangspunkt für die erste Bestandsaufnahme dient der **Serverraum**, in dem bereits zahlreiche Sensoren zur Überwachung geschäftskritischer IT-Ausrüstung im Einsatz sind.

Bevor die physische Implementierung des Cyber-Physischen Systems (CPS) umgesetzt wird, soll vorab ein **Prototyp** zur Funktionalitätsprüfung mittels vorhandener Software in Betrieb genommen werden.

2. Projektbeschreibung & Zielsetzung

Der **Teil 1** des Projekts dient der Überprüfung der grundlegenden Sensorfunktionalität und Datenaufbereitung unter Verwendung eines ESP32 als Grundlage des CPS.

Sensorwerte erfassen

Erfassen von Temperatur, Luftfeuchtigkeit (DHT22), Bewegung (AM312 PIR) und Helligkeit (LM393).

Datenverarbeitung

Aufbereitung der Sensorwerte zur **sekündlichen Übermittlung**.

Datenformat

Ablegen der Sensorwerte im **JSON-Datenformat** für die spätere Auswertung.

Lokale Signalisierung

Signalisierung der Bewegungserkennung über eine **LED**.

Lokale Visualisierung

Anzeige von Temperatur und Luftfeuchtigkeit auf einem **OLED Display (SSD1306)**.

3. Arbeitsschritte

Die folgenden Arbeitsschritte wurden gemäß Arbeitsauftrag durchgeführt:

1. Einlesen und Verarbeiten von Temperatur und Luftfeuchtigkeit mit einem DHT22-Sensor auf dem ESP3216.
 2. Speichern der Sensorwerte im JSON-Datenformat für die sekundliche Übermittlung.
 3. Erweiterung des ESP32 um einen Bewegungsmelder und einen Fotowiderstand (LM393), deren Daten ebenfalls zum JSON-Syntax hinzugefügt wurden.
 4. Signalisierung des Bewegungsmelders über eine separate LED.
 5. Visualisierung der Temperatur und Luftfeuchtigkeit auf einem OLED-Display (SSD1306).
-

4. Hardware & Komponenten

Mikrocontroller

- ESP32 DevKit
- Herzstück des CPS

Temperatur/Feuchte

- DHT22
- GPIO-Pin 19
- Messung von Temperatur und relativer Luftfeuchtigkeit

Bewegungsmelder

- AM312 PIR
- GPIO-Pin 18
- Erfassung von Bewegung (Digital)

Bewegungs-LED

- LED
- GPIO-Pin 23
- Optische Signalisierung der Bewegungserkennung

Helligkeitssensor

- LM393 (Fotowiderstand)
- ADC-Pin 34
- Analoge Messung der Lichtstärke

Display

- SSD1306 OLED (128x64)
- SCL-Pin 22
- SDA-Pin 21
- Lokale Visualisierung von Messwerten

Temperatur LED

- Rote LED - GPIO-Pin 12
 - Gelbe LED - GPIO-Pin 13
 - Grüne LED - GPIO-Pin 14
-

5. Implementierung & Programmcode

Die Implementierung erfolgte in **MicroPython**.

Alle Library's sind standardmäßig Teil von MicroPython,

bis auf die Library zur Interaktion mit dem SSD1306 OLED Display. Diese musste manuell über das Thonny IDE auf den ESP32 gespeichert werden.

5.1 Einlesen von Temperatur & Luftfeuchtigkeit (DHT22)

Der DHT22-Sensor wird über Pin 19 ausgelesen. Die Werte werden in der Funktion `get_sensors_results()` über `dht22_sensor.measure()` abgerufen.

```
dht22_sensor = DHT22(Pin(19))
# ...
def get_sensors_results():
    dht22_sensor.measure()
    temp = dht22_sensor.temperature()
    humid = dht22_sensor.humidity()
    # ...
```

5.2 Bewegungsmelder (PIR) & LED-Signalisierung

Der PIR-Sensor liefert einen digitalen Wert (High/Low) über Pin 18. Der Boolean-Wert wird in das Dictionary aufgenommen. Gleichzeitig wird die LED an Pin 23 geschaltet, um die Bewegungserkennung zu signalisieren.

```
am312_pir = Pin(18, Pin.IN)
motion_detector_led = Pin(23, Pin.OUT, drive=Pin.DRIVE_0)
# ...
movement = bool(am312_pir.value())
# ...
if movement:
    motion_detector_led.value(1)
else:
    motion_detector_led.value(0)
```

5.3 Lichtsensor (Fotowiderstand / LM393)

Der analoge Helligkeitswert wird über Pin 34 eingelesen und in der Funktion `get_lux()` unter Verwendung der Parameter **Gamma** (0.7) und **RL10** (50) in einen Lux-Wert umgerechnet.

$$\text{Lux} = \left(\frac{\text{RL10} \cdot 10^3 \cdot 10^{\text{Gamma}}}{\text{Resistance}} \right)^{\frac{1}{\text{Gamma}}}$$

```
lm393_pico = ADC(Pin(34, Pin.IN))
# ...
def get_lux():
    gamma = 0.7
    r110 = 50

    analog_value = lm393_pico.read() / 4
    voltage = analog_value / 1024 * 5
    resistance = 2000 * voltage / (1 - voltage / 5)
    return pow(r110 * 1e3 * pow(10, gamma) / resistance, (1 / gamma))
```

5.4 JSON-Datenformat & Sekündliche Ausgabe

Alle Sensorwerte werden in einem Python-Dictionary gesammelt und anschließend in einen JSON-String serialisiert (`json.dumps()`). Die Ausgabe erfolgt **sekündlich** (`sleep(1)`) über die serielle Schnittstelle (`print(json_results)`).

JSON-Inhalt

Temperatur:

"temperature"

Temperatur in Grad Celsius

Luftfeuchtigkeit:

"humidity"

Relative Luftfeuchtigkeit in Prozent

Bewegung

"movement"

Bewegung erkannt oder nicht

Helligkeit

"luminance"

Helligkeit in Lux

Beispielausgabe:

```
{
  "temperature": 22.9,
  "humidity": 55.2,
  "movement": false,
  "luminance": 118.44
}
```

5.5 OLED-Display-Ausgabe (SSD1306)

Das OLED-Display (SSD1306) wird über die I2C-Schnittstelle (SCL: Pin 22, SDA: Pin 21) angesteuert und zur lokalen Visualisierung der **Temperatur** und der **Luftfeuchtigkeit** genutzt. Die Anzeige wird sekundlich aktualisiert (`oled.fill(0)` und `oled.show()`).

```
# ...
oled.text("Temperature:", 0, 0)
oled.text(str(dict_results["temperature"]), 0, 10)
oled.text("Humidity:", 0, 25)
oled.text(str(dict_results["humidity"]), 0, 35)
oled.show()
sleep(1)
```

5.6 Temperatur LED-Anzeige

Anhand der **Temperatur** leuchtet eine Rote ($\geq 30^{\circ}\text{C}$), Gelbe ($\geq 25^{\circ}\text{C}$ und $< 30^{\circ}\text{C}$) oder Grüne ($\geq 20^{\circ}\text{C}$) LED.

```
# ...
    if temp >= 30:
        temp_led_red.value(1)
    elif temp >= 25 and temp < 30:
        temp_led_yellow(1)
    elif temp >= 20:
        temp_led_green.value(1)
    else:
        oled.fill(0)
        oled.text("Temperature Error", 0, 0)
        oled.text("Are you currently in Antarctica ?", 0, 10)
        oled.show()
```

6. Gesamter Programmcode

Der vollständige MicroPython-Code :

```
from machine import Pin, ADC, SoftI2C
import ssd1306
from time import sleep
from dht import DHT22
import json
import sys
import select

dht22_sensor = DHT22(Pin(19))
oled_width = 128
oled_height = 64
oled = ssd1306.SSD1306_I2C(oled_width, oled_height, SoftI2C(scl=Pin(22))

def get_lux():
    #Code wurde auf mehrere Zeilen aufgeteilt für lesbarkeit in Dokumer
    return pow(50 * 1e3 * pow(10, 0.7) / (2000 * ((ADC(Pin(34, Pin.IN))
    / 1024 * 5) / (1 - ((ADC(Pin(34, Pin.IN)).read() / 4) / 1024 * 5)
    / 5)), (1 / 0.7))

def display_temp_leds(temp):
    if temp >= 30:
        Pin(12, Pin.OUT).value(1)
    elif temp >= 25 and temp < 30:
        Pin(13, Pin.OUT).value(1)
    elif temp >= 20:
        Pin(14, Pin.OUT).value(1)
    else:
        oled.fill(0)
        oled.text("Temperature Error", 0, 0)
        oled.text("Are you currently in Antarctica ?", 0, 10)
        oled.show()

def get_sensors_results():
    dht22_sensor.measure()
    temp = dht22_sensor.temperature()
    humid = dht22_sensor.humidity()
```

```

movement = bool(Pin(18, Pin.IN).value())
luminance = get_lux()

if movement:
    Pin(23, Pin.OUT, drive=Pin.DRIVE_0).value(1)
else:
    Pin(23, Pin.OUT, drive=Pin.DRIVE_0).value(0)

display_temp_leds(temp)

dict_results = {
    "temperature": temp,
    "humidity": humid,
    "movement": movement,
    "luminance": luminance
}
return dict_results

while True:
    if select.select([sys.stdin], [], [], 0)[0]:
        line = sys.stdin.readline().strip()
        if line == "exit":
            oled.fill(0)
            sys.exit()

    json_results = json.dumps(get_sensors_results())
    print(json_results)
    dict_results = get_sensors_results()

    oled.fill(0)
    oled.text("Temperature:", 0, 0)
    oled.text(str(dict_results["temperature"]), 0, 10)
    oled.text("Humidity:", 0, 25)
    oled.text(str(dict_results["humidity"]), 0, 35)
    oled.show()
    sleep(1)

```

7. Testergebnisse & Fazit

Testergebnisse

- **DHT22:** Temperatur- und Feuchtigkeitsmessung erfolgreich.
- **PIR & LED:** Der Bewegungsmelder liefert korrekte Zustände, die LED signalisiert zuverlässig Bewegung.
- **LM393:** Der Helligkeitswert wird erfasst und in Lux umgerechnet.
- **JSON-Ausgabe:** Das korrekte JSON-Format wird sekundlich über die serielle Schnittstelle ausgegeben.
- **SSD1306:** Temperatur und Luftfeuchtigkeit werden lokal auf dem OLED-Display visualisiert.
- **Temperatur-LEDs:** Das Level der Temperatur wird erfolgreich durch eine der drei LEDs dargestellt.

Beispiel für eine erfolgreiche JSON-Ausgabe:

```
{  
  "temperature": 22.9,  
  "humidity": 55.2,  
  "movement": false,  
  "luminance": 118.44  
}
```

Fazit

Der erste Projektteil zur Erstellung eines funktionsfähigen Prototyps für die Sensorik und Datenaufbereitung mit dem ESP32 wurde erfolgreich abgeschlossen. Die Grundfunktionalität (Sensordatenerfassung, JSON-Formatierung, lokale Anzeige) ist gegeben. Damit ist die Basis geschaffen, um das CPS in das interne Netzwerk zu integrieren und im nächsten Schritt die Datenübertragung über MQTT an einen MQTT-Client zur Visualisierung auf einem Dashboard umzusetzen.